

Quy hoạch tuyến tính và thuật toán đơn hình

Wu Yifan

Khối trung học phổ thông, Trường Phổ thông trực thuộc Đại học Cát Lâm

3 tháng 3, 2015

Nguồn gốc: Linear Programming and Simplex Algorithm

Mathematics / Linear programming and simplex algorithm - Wu Yifan

Abstract

Bài giảng nhắc lại dạng chuẩn của quy hoạch tuyến tính, cách đưa một mô hình tuyến tính bất kỳ về dạng chuẩn và dạng lỏng, rồi trình bày thuật toán đơn hình qua khái niệm biến cơ sở, biến phi cơ sở, nghiệm khả thi, phép xoay trục và quy tắc Bland. Phần cuối nêu một số ứng dụng trong bài toán thi lập trình, đặc biệt là mối liên hệ với mô hình luồng mạng, đối ngẫu tuyến tính và ví dụ BZOJ3876.

Contents

1	Giới thiệu	1
2	Dạng chuẩn	1
2.1	Hàm mục tiêu	2
2.2	Chiều của bất đẳng thức	2
2.3	Ràng buộc trên từng biến	2
3	Dạng lỏng	2
4	Một số định nghĩa	2
5	Thuật toán đơn hình	3
5.1	Ví dụ xoay trục	3
5.2	Mã xoay trục	4
6	Ứng dụng	4
7	Ví dụ BZOJ3876	4
7.1	Đối ngẫu tuyến tính	5
8	Kết luận	5

1 Giới thiệu

Ở bậc phổ thông, nhiều người đã gặp quy hoạch tuyến tính và biết đại khái nó là gì: có một số biến, cần tìm giá trị tốt nhất của một hàm theo các biến đó dưới một số ràng buộc. Cả ràng buộc lẫn hàm mục tiêu đều tuyến tính.

Nếu các bài quy hoạch tuyến tính gặp trong chương trình phổ thông đều được giải bằng hình học, thì cách đó trở nên rất khó chịu khi số chiều tăng lên. Vì vậy ta dùng thuật toán đơn hình để xử lý trực tiếp hơn. Các chứng minh chi tiết trong bài giảng gốc được giữ ở mức trực giác.

2 Dạng chuẩn

Ta định nghĩa một dạng chuẩn cho quy hoạch tuyến tính:

$$\begin{array}{ll}\text{Maximize} & c^T x, \\ \text{S.T.} & Ax \leq b, \quad x \geq 0.\end{array}$$

Ở đây giả sử có n biến và m ràng buộc. Vector x là vector cột chứa các biến; c là vector hệ số của hàm mục tiêu; A là ma trận hệ số của các ràng buộc; b là vector cột hằng số ở vế phải. Điều kiện $x \geq 0$ nghĩa là mọi thành phần của x đều không âm. Kích thước tương ứng là

$$c^T[1, n], \quad x[n, 1], \quad A[m, n], \quad b[m, 1].$$

Tất nhiên không phải mọi bài toán quy hoạch tuyến tính ban đầu đều đã ở dạng này. Ta lần lượt xét các phép biến đổi đưa mô hình tổng quát về dạng chuẩn, và các phép biến đổi này không làm thay đổi giá trị tối ưu cần tìm.

2.1 Hàm mục tiêu

Nếu bài toán yêu cầu cực tiểu thay vì cực đại, ta đổi dấu toàn bộ hệ số của hàm mục tiêu, giải bài toán cực đại mới, rồi đổi dấu đáp án. Nói cách khác,

$$\min c^T x = -\max(-c)^T x.$$

2.2 Chiều của bất đẳng thức

Trong dạng chuẩn, mọi ràng buộc đều có dạng $\leq$. Nếu ràng buộc có dạng $\geq$, ta nhân cả hai vế với -1 để đổi về $\leq$. Nếu ràng buộc là đẳng thức, chẳng hạn $a = b$, ta thay bằng hai ràng buộc

$$a \leq b, \quad b \leq a.$$

2.3 Ràng buộc trên từng biến

Mỗi biến trong dạng chuẩn phải không âm. Với các ràng buộc riêng trên một biến, ta thay biến như sau.

Nếu $x \leq a$, đặt

$$x' = a - x, \quad x' \geq 0.$$

Nếu $x \geq a$, đặt

$$x' = x - a, \quad x' \geq 0.$$

Nếu x không bị chặn dấu, đặt

$$x = x' - x'', \quad x' \geq 0, \quad x'' \geq 0.$$

Sau các phép thay thế trên, mọi quy hoạch tuyến tính đều có thể chuyển về dạng chuẩn.

3 Dạng lỏng

Để áp dụng đơn hình thuận tiện hơn, ta đổi các bất đẳng thức trong dạng chuẩn thành đẳng thức bằng cách thêm biến phụ. Với ràng buộc thứ i :

$$\sum_{j=1}^n A_{i,j}x_j \leq b_i,$$

thêm biến x_{n+i} và viết

$$x_{n+i} + \sum_{j=1}^n A_{i,j}x_j = b_i, \quad x_{n+i} \geq 0.$$

Như vậy mọi ràng buộc đã trở thành đẳng thức. Tổng cộng có $n + m$ biến và mọi biến vẫn không âm. Thuật toán đơn hình sẽ được mô tả trên dạng lỏng này.

4 Một số định nghĩa

Biến cơ sở là biến được viết đầu tiên trong mỗi phương trình ràng buộc. Trong dạng lỏng ban đầu, $x_{n+1}, \dots, x_{n+m}$ là các biến cơ sở, nên có đúng m biến cơ sở.

Biến phi cơ sở là các biến còn lại. Ban đầu $x_1, \dots, x_n$ là biến phi cơ sở, nên có đúng n biến phi cơ sở. Hàm mục tiêu chỉ liên quan trực tiếp đến các biến phi cơ sở.

Ký hiệu tập biến cơ sở là B , tập biến phi cơ sở là N . Hai tập này thay đổi trong quá trình thuật toán.

Nghiệm khả thi là một bộ giá trị làm thỏa mãn tất cả ràng buộc. **Nghiệm tối ưu** là nghiệm khả thi làm hàm mục tiêu đạt giá trị tốt nhất. Mục tiêu của quy hoạch tuyến tính là tìm một nghiệm tối ưu.

5 Thuật toán đơn hình

Trước hết giả sử nghiệm cơ sở hiện tại là khả thi, tức là khi mọi biến phi cơ sở bằng 0, các biến cơ sở thu được đều không âm. Trường hợp không có tính chất này sẽ được xét sau.

Nếu tồn tại một biến phi cơ sở có hệ số dương trong hàm mục tiêu, tăng biến đó có thể làm giá trị mục tiêu tăng. Nếu mọi hệ số của biến phi cơ sở đều không dương, nghiệm hiện tại đã tối ưu: khi các biến phi cơ sở không âm, không thể tăng mục tiêu thêm. Lúc này giá trị tối ưu chính là hằng số còn lại trong hàm mục tiêu.

Giả sử ta chọn biến phi cơ sở thứ e , ký hiệu x_{N_e} , để tăng. Biến này không thể tăng vô hạn nếu một ràng buộc trở nên chặt trước. Xét ràng buộc thứ i :

$$x_{B_i} + \sum_{j=1}^n A_{i,j}x_{N_j} = b_i.$$

Nếu $A_{i,e} > 0$, để giữ $x_{B_i} \geq 0$ ta cần

$$x_{N_e} \leq \frac{b_i}{A_{i,e}}.$$

Ta chọn ràng buộc chặt nhất. Giả sử đó là ràng buộc thứ l , tương ứng với biến cơ sở x_{B_l} . Nếu không có ràng buộc nào như vậy, tức giá trị nhỏ nhất là INF, thì x_{N_e} có thể tăng vô hạn và bài toán không bị chặn trên.

Vì ràng buộc thứ l là chặt nhất, đẳng thức tương ứng thật sự đạt dấu bằng. Ta thực hiện phép xoay trục: giải phương trình đó theo x_{N_e} , rồi thay x_{N_e} vào mọi phương trình khác và vào hàm mục tiêu. Khi đó x_{B_l} trở thành biến phi cơ sở thứ e , còn x_{N_e} trở thành biến cơ sở thứ l . Nói cách khác, hai biến này đổi vai trò cho nhau, và cả hai tập B, N đều thay đổi.

5.1 Ví dụ xoay trục

Xét quy hoạch tuyến tính:

$$\begin{array}{ll}\text{Maximize} & 3x_1 + x_2 + 2x_3, \\ \text{S.T.} & x_4 + x_1 + x_2 + 3x_3 = 30, \\ & x_5 + 2x_1 + 2x_2 + 5x_3 = 24, \\ & x_6 + 4x_1 + x_2 + 2x_3 = 36.\end{array}$$

Ban đầu

$$B = \{4, 5, 6\}, \quad N = \{1, 2, 3\}.$$

Ta chọn biến phi cơ sở x_1 để tăng. Từ ba ràng buộc nhận được lần lượt

$$x_1 \leq 30, \quad x_1 \leq 12, \quad x_1 \leq 9.$$

Ràng buộc thứ ba chặt nhất, nên ta đổi biến cơ sở thứ ba với biến phi cơ sở thứ nhất. Từ ràng buộc thứ ba:

$$x_1 = 9 - \frac{x_6}{4} - \frac{x_2}{4} - \frac{x_3}{2}.$$

Thay biểu thức này vào những nơi còn lại, ta được quy hoạch tuyến tính mới:

$$\begin{array}{ll}\text{Maximize} & -\frac{3x_6}{4} + \frac{x_2}{4} + \frac{x_3}{2} + 27, \\ \text{S.T.} & x_4 - \frac{x_6}{4} + \frac{3x_2}{4} + \frac{5x_3}{2} = 21, \\ & x_5 - \frac{x_6}{2} + \frac{3x_2}{2} + 4x_3 = 6, \\ & x_1 + \frac{x_6}{4} + \frac{x_2}{4} + \frac{x_3}{2} = 9.\end{array}$$

Lúc này

$$B = \{4, 5, 1\}, \quad N = \{6, 2, 3\}.$$

Hằng số trong hàm mục tiêu đã tăng. Lập quá trình này, thuật toán sẽ tìm được một nghiệm tối ưu hoặc kết luận miền khả thi không bị chặn theo hướng cực đại.

5.2 Mã xoay trục

Đoạn mã sau giữ nguyên tên biến trong slide:

```
inline void pivot(int l,int e){
    int i,j,k;
    b[l]/=A[l][e];
    for(i=1;i<=n;++i)if(i!=e)A[l][i]/=A[l][e];
    A[l][e]=1/A[l][e];

    for(i=1;i<=m;++i)if(i!=l&&fabs(A[i][e])>eps){
        b[i]-=A[i][e]*b[l];
        for(j=1;j<=n;++j)if(j!=e)A[i][j]-=A[i][e]*A[l][j];
        A[i][e]-=A[i][e]*A[l][e];
    }

    v+=c[e]*b[l];
```

```

for(i=1;i<=n;++i)if(i!=e)c[i]-=c[e]*A[l][i];
c[e]=-c[e]*A[l][e];

swap(B[l],N[e]);
}

```

Cần chú ý quy tắc Bland: khi chọn biến phi cơ sở đi vào cơ sở, hãy chọn biến có hệ số dương trong mục tiêu và có chỉ số nhỏ nhất. Cách chọn này tránh các vòng lặp do trường hợp suy biến.

Mỗi lần xoay trục có độ phức tạp $\mathcal{O}(nm)$. Thuật toán đơn hình không phải thuật toán thời gian đa thức theo nghĩa chặt, và vẫn có thể bị dữ liệu đặc biệt làm chậm đến mức mũ. Tuy vậy trong thực hành lập trình thi đấu, nếu dữ liệu không cố ý khai thác điểm yếu này, nó thường chạy đủ tốt.

6 Ứng dụng

Thuật toán đơn hình dùng để làm gì? Câu trả lời thực dụng trong slide là: giải bài tập. Tuy nhiên trước các bài quy hoạch tuyến tính mạnh, những bài chỉ yêu cầu áp dụng trực tiếp không nhiều.

Một nguồn mô hình quan trọng là luồng mạng. Các bài như luồng cực đại, luồng chi phí nhỏ nhất, luồng có chặn trên và chặn dưới đều có thể xem là quy hoạch tuyến tính. Do giới hạn thời gian, slide chỉ nhắc hướng này và gợi ý tham khảo tài liệu “Quy hoạch tuyến tính và luồng mạng” của Cao Qinxiang.

7 Ví dụ BZOJ3876

Xét bài BZOJ3876, Ahoi2014, tên thường dịch là “Cốt truyện nhánh”. Nếu bỏ qua mô hình luồng mạng, ta có thể dùng quy hoạch tuyến tính trực tiếp.

Gọi x_i là số lần cạnh thứ i được đi qua. Thêm một siêu nguồn nối vào đỉnh 1, và mỗi đỉnh nối đến siêu đích. Với mọi đỉnh trung gian, số lần đi vào bằng số lần đi ra. Ngoài ra $x_i \geq 1$ đối với các cạnh gốc của bài toán; điều kiện này không áp dụng cho các cạnh thêm vào.

Ta có thể đặt từng ràng buộc riêng vào hệ lớn. Hàm mục tiêu cũng rất rõ ràng: cần cực tiểu tổng chi phí. Đổi dấu hệ số để biến thành bài cực đại. Vấn đề là một số ràng buộc có dạng ≥ 1 ; sau khi đổi dấu chúng thành ≤ -1 , nghiệm cơ sở ban đầu không còn khả thi.

7.1 Đối ngẫu tuyến tính

Với bài toán

$$\begin{array}{ll} \text{Maximize} & c^T x, \\ \text{S.T.} & Ax \leq b, \quad x \geq 0, \end{array}$$

bài toán đối ngẫu là

$$\begin{array}{ll} \text{Minimize} & b^T y, \\ \text{S.T.} & A^T y \geq c, \quad y \geq 0. \end{array}$$

Hai bài toán có cùng giá trị tối ưu. Vì vậy ta có thể đối ngẫu trực tiếp mô hình ban đầu để thu được một bài toán cực đại có nghiệm cơ sở ban đầu khả thi, khỏi phải tự xây dựng nghiệm khả thi ban đầu.

Cách làm này vẫn có thể quá chậm. Trong mô hình trực tiếp, số biến và số ràng buộc đều là $\mathcal{O}(n+m)$; với bài này thời gian chạy được nhận xét là khoảng ba lần ZKW chi phí nhỏ nhất, nên không thật sự phù hợp.

Điểm cần sửa là không đặt quá nhiều ràng buộc đơn lẻ cho từng phần tử. Ta có thể gộp chúng vào các ràng buộc tổng, dùng biến thay thế dạng $x' = x - 1$, để số ràng buộc của bài gốc còn $\mathcal{O}(n)$. Sau khi

đổi ngẫu, hệ có $\mathcal{O}(n)$ biến và $\mathcal{O}(n + m)$ ràng buộc, gần với mức có thể chấp nhận. Nhược điểm là ở bước này có thể lại mất nghiệm khả thi ban đầu, nên cần xử lý cẩn thận theo từng mô hình.

8 Kết luận

Quy hoạch tuyến tính có nhiều ứng dụng. Thuật toán đơn hình có cài đặt đơn giản, độ phức tạp thực tế thường thấp và không dễ bị chặn bằng dữ liệu thông thường; vì vậy nó là một thuật toán đáng cân nhắc trong thi lập trình.

Các bài gợi ý:

BZOJ1061, BZOJ3112, BZOJ3265, BZOJ3876.